Finance Module — Postman Testing Reference

Base URL: http://localhost:3000 | Content-Type: application/json | Test in this order — each section depends on IDs from the previous one. Copy IDs from responses as you go.

After each request, check: success=true, correct HTTP status, and the specific fields listed under each endpoint.

1. Modules /api/modules

POST `/api/modules`

```
{
"courseCode": "DICT",
"courseName": "Diploma in ICT",
"level": 4,
"moduleNum": 1
}
```

✓ status 201
✓ response has id starting with MOD-
✓ Copy this id — you need it for fee structures and enrollments

POST `/api/modules`

```
{
"courseCode": "ICTP",
"courseName": "Computer Packages",
"level": 3,
"moduleNum": 1
}
```

✓ status 201 — creates the packages module
✓ Copy this id as PACKAGES_MODULE_ID

POST `/api/modules`

```
{
"courseCode": "DICT",
"courseName": "Diploma in ICT",
"level": 4,
"moduleNum": 1
}
```

✓ status 409 — duplicate courseCode+level+moduleNum must be rejected

GET `/api/modules`

✓ status 200
✓ returns array, both modules visible

GET `/api/modules?courseCode=DICT`

✓ status 200
✓ all items in array have courseCode = DICT

GET `/api/modules/{id}`

✓ status 200, returns single module

GET `/api/modules/FAKEID`

✓ status 404

PATCH `/api/modules/{id}/deactivate`

✓ status 200, isActive = false
✓ GET /api/modules after this — deactivated module no longer appears

2. Voteheads /api/voteheads

Create all voteheads you need. Copy each id — you will need them in fee structures.

POST /api/voteheads

```
{
"name": "Admission Fee",
"isOneTime": true,
"isRefundable": false,
"priorityOrder": 1
}
```

✓ status 201 — copy id as VH-Admission

POST /api/voteheads

```
{
"name": "ID Card",
"isOneTime": true,
"isRefundable": false,
"priorityOrder": 2
}
```

✓ status 201 — copy id as VH-ID

POST /api/voteheads

```
{
"name": "Caution Fee",
"isOneTime": true,
"isRefundable": true,
"priorityOrder": 3
}
```

✓ status 201 — copy id as VH-Caution

POST /api/voteheads

```
{
"name": "Tuition",
"isOneTime": false,
"isRefundable": false,
"priorityOrder": 4
}
```

✓ status 201 — copy id as VH-Tuition

POST /api/voteheads

```
{
"name": "PE",
"isOneTime": false,
"isRefundable": false,
"priorityOrder": 5
}
```

✓ status 201 — copy id as VH-PE

POST /api/voteheads

```
{
"name": "Medical & Insurance",
"isOneTime": false,
"isRefundable": false,
"priorityOrder": 6
}
```

✓ status 201 — copy id as VH-Medical

POST /api/voteheads

```
{
"name": "Examination Fee",
"isOneTime": false,
"isRefundable": false,
"priorityOrder": 7
}
```

✓ status 201 — copy id as VH-Exam

POST /api/voteheads

```
{
"name": "Tuition",
"isOneTime": false,
"isRefundable": false,
"priorityOrder": 99
}
```

✓ status 409 — duplicate name must be rejected

POST `/api/voteheads`

```
{

"name": "Some Fee",

"isOneTime": false,

"isRefundable": false,

"priorityOrder": 4

}
```

✓ status 409 — duplicate priorityOrder must be rejected

GET `/api/voteheads`

✓ status 200
✓ array sorted by priorityOrder ascending
✓ Admission Fee appears before Tuition

PATCH `/api/voteheads/{id}`

```
{

"description": "Updated description"

}
```

✓ status 200, description updated

3. Fee Structures /api/fee-structures

Use the module id and votehead ids copied from sections 1 and 2.

POST `/api/fee-structures`

```
{
"name": "DICT L4 Mod1 Regular 2025",
"moduleId": "{DICT_MODULE_ID}",
"studentCategory": "regular",
"validFrom": "2025-01-01",
"validTo": "2025-12-31",
"createdBy": "USR001",
"lines": [
{ "voteheadId": "{VH-Admission}", "amount": 500, "priorityOrder": 1 },
{ "voteheadId": "{VH-ID}", "amount": 500, "priorityOrder": 2 },
{ "voteheadId": "{VH-Caution}", "amount": 1000, "priorityOrder": 3 },
{ "voteheadId": "{VH-Tuition}", "amount": 15000, "priorityOrder": 4 },
{ "voteheadId": "{VH-PE}", "amount": 5250, "priorityOrder": 5 },
{ "voteheadId": "{VH-Medical}", "amount": 2400, "priorityOrder": 6 },
{ "voteheadId": "{VH-Exam}", "amount": 3000, "priorityOrder": 7 }
]
}
```

✓ status 201
✓ totalAmount = 27650 (sum of all line amounts — computed server side)
✓ response.lines has 7 items, each with votehead name included
✓ Copy id as FS_REGULAR_2025

POST `/api/fee-structures`

```
{
"name": "DICT L4 Mod1 Regular 2030",
"moduleId": "{DICT_MODULE_ID}",
"studentCategory": "regular",
"validFrom": "2030-01-01",
"validTo": null,
"createdBy": "USR001",
"lines": [
```

```
{ "voteheadId": "{VH-Tuition}", "amount": 25000, "priorityOrder": 1 },

{ "voteheadId": "{VH-Exam}", "amount": 5000, "priorityOrder": 2 }

]

}
```

✓ status 201 — creates 2030 structure for same module
✓ totalAmount = 30000
✓ Copy id as FS_REGULAR_2030

POST `/api/fee-structures`

```
{

"name": "Packages 2025",

"moduleId": "{PACKAGES_MODULE_ID}",

"studentCategory": "short_course",

"validFrom": "2025-01-01",

"validTo": null,

"createdBy": "USR001",

"lines": [

{ "voteheadId": "{VH-Tuition}", "amount": 4500, "priorityOrder": 1 }

]

}
```

✓ status 201 — single-votehead structure for packages
✓ Copy id as FS_PACKAGES

POST `/api/fee-structures`

```
{

"name": "Bad structure",

"moduleId": "{DICT_MODULE_ID}",

"studentCategory": "regular",

"validFrom": "2025-01-01",

"createdBy": "USR001",

"lines": []

}
```

✓ status 400 — empty lines must be rejected

GET `/api/fee-structures/resolve?moduleId={DICT_MODULE_ID}&studentCategory;=regular&enrollmentDate;=2025-06-15`

✓ status 200 — resolves FS_REGULAR_2025
✓ validFrom <= 2025-06-15 <= validTo

GET	`/api/fee-structures/resolve?moduleId={DICT_MODULE_ID}&studentCategory;=regular&enrollmentDate;=2030-03-15`

✓ status 200 — resolves FS_REGULAR_2030 (validTo=null)
✓ totalAmount = 30000, NOT 27650
✓ This proves enrollment date drives fee resolution

GET	`/api/fee-structures/resolve?moduleId={DICT_MODULE_ID}&studentCategory;=nita&enrollmentDate;=2025-06-15`

✓ status 404 — no NITA structure exists for this module

GET	`/api/fee-structures/resolve?moduleId={DICT_MODULE_ID}&studentCategory;=regular&enrollmentDate;=2026-01-01`

✓ status 404 — date is outside FS_REGULAR_2025 range (validTo 2025-12-31) and before FS_REGULAR_2030 (validFrom 2030-01-01)

4. Enrollments /api/enrollments

Each enrollment atomically creates one obligation. Check both in the response.

POST `/api/enrollments`

```
{
"studentId": "S12345",
"moduleId": "{DICT_MODULE_ID}",
"studentCategory": "regular",
"enrollmentDate": "2025-01-06",
"enrolledBy": "USR001"
}
```

✓ status 201
✓ response.enrollment.id exists — copy as ENR001
✓ response.obligation.id exists — copy as OBL001
✓ response.obligation.totalAmount = 27650
✓ response.obligation.totalPaid = 0
✓ response.obligation.totalBalance = 27650
✓ response.obligation.status = "unpaid"

POST `/api/enrollments`

```
{
"studentId": "S12345",
"moduleId": "{PACKAGES_MODULE_ID}",
"studentCategory": "short_course",
"enrollmentDate": "2025-01-06",
"enrolledBy": "USR001"
}
```

✓ status 201 — same student, different module, at same time
✓ response.obligation.totalAmount = 4500
✓ Copy obligation id as OBL002 — packages obligation

POST `/api/enrollments`

```
{
"studentId": "S12345",
"moduleId": "{DICT_MODULE_ID}",
"studentCategory": "regular",
"enrollmentDate": "2025-01-06",
"enrolledBy": "USR001"
}
```

✓ status 409 — student already enrolled in this module

POST `/api/enrollments`

```
{
"studentId": "S12345",
"moduleId": "{DICT_MODULE_ID}",
"studentCategory": "nita",
"enrollmentDate": "2025-01-06",
"enrolledBy": "USR001"
}
```

✓ status 404 — no fee structure for NITA on this module

GET `/api/enrollments/{ENR001}`

✓ status 200
✓ includes module info, fee structure with lines, obligation

GET `/api/enrollments/student/S12345`

✓ status 200
✓ returns array of 2 enrollments (DICT + Packages)

PATCH `/api/enrollments/{ENR001}/status`

```
{ "status": "completed" }
```

✓ status 200, enrollment.status = "completed"

5. Payments /api/payments

Use OBL001 (27650 total) and OBL002 (4500 total) from section 4.

5A — Basic payment against one obligation

POST `/api/payments`

```
{
"studentId": "S12345",
"obligations": [{ "obligationId": "{OBL001}", "amount": 5000 }],
"totalAmount": 5000,
"paymentMode": "mpesa",
"transactionRef": "QXZ123ABC",
"fundingSource": "self",
"receivedBy": "USR001",
"paymentDate": "2025-01-15"
}
```

✓ status 201
✓ receiptNumber starts with RCP-
✓ Copy payment id as PMT001
✓ Now GET OBL001 from DB or re-enroll check — totalPaid must be 5000
✓ totalBalance must be 22650
✓ status must be "partial"
✓ payment_distributions table must be EMPTY for this obligation at this point

POST `/api/payments`

```
{
"studentId": "S12345",
"obligations": [{ "obligationId": "{OBL001}", "amount": 10000 }],
"totalAmount": 10000,
"paymentMode": "cash",
"fundingSource": "self",
"receivedBy": "USR001",
"paymentDate": "2025-01-20"
}
```

✓ status 201 — second payment
✓ OBL001 totalPaid now 15000, totalBalance 12650, status "partial"

5B — Pay packages (OBL002) in full

POST `/api/payments`

```
{
"studentId": "S12345",
"obligations": [{ "obligationId": "{OBL002}", "amount": 4500 }],
"totalAmount": 4500,
"paymentMode": "cash",
"fundingSource": "self",
"receivedBy": "USR001",
"paymentDate": "2025-01-20"
}
```

✓ status 201
✓ OBL002 totalPaid = 4500, totalBalance = 0, status = "cleared"

5C — Pay two obligations in one payment call

POST /api/payments

```
{
"studentId": "S12345",
"obligations": [
{ "obligationId": "{OBL001}", "amount": 7000 },
{ "obligationId": "{OBL002}", "amount": 1000 }
],
"totalAmount": 8000,
"paymentMode": "bank_transfer",
"transactionRef": "KCB-REF-9900",
"fundingSource": "kuccps",
"receivedBy": "USR001",
"paymentDate": "2025-01-22"
}
```

✓ status 201 — single payment covering two obligations
✓ OBL001 balance reduces by 7000
✓ OBL002 balance reduces by 1000 (it was already cleared — status becomes overpaid)

5D — Error cases

POST /api/payments

```
{
"studentId": "S12345",
```

```
"obligations": [{ "obligationId": "{OBL001}", "amount": 3000 }],
"totalAmount": 5000,
"paymentMode": "cash",
"fundingSource": "self",
"receivedBy": "USR001",
"paymentDate": "2025-01-23"
}
```

✓ status 400 — obligation amounts (3000) do not sum to totalAmount (5000)

POST `/api/payments`

```
{
"studentId": "S12345",
"obligations": [{ "obligationId": "{OBL001}", "amount": 1000 }],
"totalAmount": 1000,
"paymentMode": "mpesa",
"transactionRef": "QXZ123ABC",
"fundingSource": "self",
"receivedBy": "USR001",
"paymentDate": "2025-01-24"
}
```

✓ status 409 — duplicate transactionRef QXZ123ABC

POST `/api/payments`

```
{
"studentId": "S99999",
"obligations": [{ "obligationId": "{OBL001}", "amount": 1000 }],
"totalAmount": 1000,
"paymentMode": "cash",
"fundingSource": "self",
"receivedBy": "USR001",
"paymentDate": "2025-01-25"
}
```

✓ status 403 — OBL001 belongs to S12345, not S99999

5E — Reversal

POST `/api/payments/{PMT001}/reverse`

```
{

"obligations": [{ "obligationId": "{OBL001}", "amount": 5000 }],

"reversedBy": "USR002"

}
```

✓ status 201
✓ response.isReversal = true
✓ receiptNumber starts with RVS-
✓ OBL001 totalPaid drops by 5000
✓ Copy reversal id as RVS001

POST `/api/payments/{RVS001}/reverse`

```
{

"obligations": [{ "obligationId": "{OBL001}", "amount": 5000 }]

}
```

✓ status 400 — cannot reverse a reversal

POST `/api/payments/{PMT001}/reverse`

```
{

"obligations": [{ "obligationId": "{OBL001}", "amount": 5000 }]

}
```

✓ status 409 — PMT001 already has a reversal

6. Receipts /api/receipts

This is where the waterfall runs. POST /generate triggers the calculation and writes distributions. GET /breakdown gives you a live view without writing anything.

Current state of OBL001 before generating receipt:

After section 5: PMT001 (5000) was reversed, PMT002 (10000) stands. Net paid on OBL001 = 10000. Fee structure has: Admission 500, ID 500, Caution 1000, Tuition 15000, PE 5250, Medical 2400, Exam 3000.

Expected waterfall: Admission 500 cleared → ID 500 cleared → Caution 1000 cleared → Tuition 8000/15000 partial → PE, Medical, Exam all unpaid.

POST `/api/receipts/{OBL001}/generate`

✓ status 200
✓ voteheadBreakdown has 7 items (one per fee structure line)
✓ Admission: amountDue=500, amountPaid=500, status="cleared"
✓ ID Card: amountDue=500, amountPaid=500, status="cleared"
✓ Caution: amountDue=1000, amountPaid=1000, status="cleared"
✓ Tuition: amountDue=15000, amountPaid=8000, amountBalance=7000, status="partial"
✓ PE: amountDue=5250, amountPaid=0, status="unpaid"
✓ Medical: amountDue=2400, amountPaid=0, status="unpaid"
✓ Exam: amountDue=3000, amountPaid=0, status="unpaid"
✓ summary.totalPaid = 10000
✓ Sum of all amountPaid in breakdown = 10000
✓ payment_distributions table now has rows for this obligation

POST `/api/receipts/{OBL001}/generate`

✓ Call again with no payments in between
✓ status 200, identical result to above
✓ Distribution rows in DB = same count (old deleted, new written — not doubled)
✓ This proves idempotency

GET `/api/receipts/{OBL001}/breakdown`

✓ status 200, same breakdown data as generate
✓ Call it multiple times — distribution row count in DB does not change
✓ This proves GET does not write anything

6A — Receipt after OBL002 (packages, fully cleared)

POST `/api/receipts/{OBL002}/generate`

✓ status 200
✓ voteheadBreakdown has 1 item (Tuition only)
✓ Tuition: amountDue=4500, amountPaid=4500, status="cleared"
✓ summary.totalPaid = 4500, totalBalance = 0

7. Refunds /api/refunds

Caution Fee (VH-Caution) has isRefundable=true. It has been paid (cleared) on OBL001. Refunds need authorisedBy and processedBy — both required.

POST `/api/refunds`

```
{
"obligationId": "{OBL001}",
"voteheadId": "{VH-Caution}",
"amount": 1000,
"paymentMode": "mpesa",
"transactionRef": "RFD-TX-001",
"authorisedBy": "USR002",
"processedBy": "USR001",
"refundDate": "2025-06-15",
"notes": "Programme completed"
}
```

✓ status 201
✓ receiptNumber starts with RCF-
✓ amount = 1000
✓ voteheadId = VH-Caution
✓ Copy refund id as RFD001

POST `/api/refunds`

```
{
"obligationId": "{OBL001}",
"voteheadId": "{VH-Tuition}",
"amount": 1000,
"paymentMode": "cash",
"authorisedBy": "USR002",
"processedBy": "USR001",
"refundDate": "2025-06-15"
}
```

✓ status 400 — Tuition is not refundable (isRefundable=false)

POST `/api/refunds`

```
{
"obligationId": "{OBL001}",
"voteheadId": "{VH-Caution}",
```

```
"amount": 99999,

"paymentMode": "cash",

"authorisedBy": "USR002",

"processedBy": "USR001",

"refundDate": "2025-06-15"

}
```

✓ status 400 — refund amount exceeds amount paid for this votehead

POST /api/refunds

```
{

"obligationId": "{OBL001}",

"voteheadId": "{VH-Caution}",

"amount": 1000,

"paymentMode": "cash",

"authorisedBy": "USR002",

"processedBy": "USR001",

"refundDate": "2025-06-16"

}
```

✓ status 409 — refund already exists for Caution on OBL001

GET /api/refunds/{RFD001}

✓ status 200, includes votehead name and obligation info

GET /api/refunds/student/S12345

✓ status 200, returns array with RFD001

8. Key Checks Summary

Area	Check
Modules	Duplicate courseCode+level+moduleNum → 409
Modules	Unknown id → 404
Voteheads	Duplicate priorityOrder → 409
Voteheads	Duplicate name → 409
Voteheads	GET returns sorted by priorityOrder ASC
Fee Structures	totalAmount computed server-side from lines sum
Fee Structures	Empty lines array → 400
Fee Structures	resolve: correct structure picked by date
Fee Structures	resolve: date outside all ranges → 404
Fee Structures	resolve: validTo=null matches any future date
Fee Structures	resolve: wrong studentCategory → 404
Enrollments	Creates obligation atomically — both exist or neither
Enrollments	obligation.totalAmount = feeStructure.totalAmount
Enrollments	obligation.totalBalance = totalAmount on creation
Enrollments	obligation.status = unpaid on creation
Enrollments	Duplicate active enrollment → 409
Enrollments	No matching fee structure → 404 (not 500)
Payments	totalPaid increments correctly after each payment
Payments	status: 0=unpaid, part=partial, full=cleared
Payments	payment_distributions table empty until receipt generated
Payments	Duplicate transactionRef → 409
Payments	Obligation amounts not summing to totalAmount → 400
Payments	Wrong student → 403
Payments	Reversal receiptNumber starts with RVS-
Payments	Reversal reduces totalPaid correctly
Payments	Reversing a reversal → 400
Payments	Double reversal of same payment → 409
Receipts	Sum of amountPaid in breakdown = obligation.totalPaid
Receipts	Voteheads filled in priority order (lower number first)
Receipts	Unpaid voteheads show amountPaid=0
Receipts	Calling generate twice — same result, no duplicate rows
Receipts	GET /breakdown does not write distribution rows
Receipts	After reversal, net=0 → all voteheads unpaid on receipt
Refunds	receiptNumber starts with RCF-

Area	Check
Refunds	Non-refundable votehead → 400
Refunds	Refund > amount paid → 400
Refunds	Votehead not paid at all → 400
Refunds	Duplicate refund same obligation+votehead → 409